

HelixAgent

HelixAgent

एक मॉडल न चुनें — उन्हें बहस करने दें, और जिस उत्तर पर वे सहमत हों उसे लागू करें।

सारांश

HelixAgent एक उत्पादन-तैयार, AI-संचालित समूह LLM सेवा है जो Go में कई भाषा मॉडलों के प्रतिक्रियाओं को बुद्धिमानी से संयोजित करती है — जिसमें बहु-चरणीय AI बहस प्रणाली और गतिशील सत्यापन-आधारित प्रदाता चयन शामिल है — ताकि सबसे सटीक और विश्वसनीय आउटपुट प्राप्त किया जा सके।

संक्षिप्त विवरण

HelixAgent एक Go-आधारित समूह LLM सेवा है जो कई प्रदाताओं को एक सटीक उत्तर में समाहित करती है। यह बहु-चरणीय AI बहस आयोजित करती है, LLMsVerifier के माध्यम से प्रदाताओं का गतिशील मूल्यांकन करती है, आत्मविश्वास-आधारित रणनीतियों के साथ रूटिंग करती है, और उत्पादन सुविधाएँ प्रदान करती है: कैशिंग, निगरानी, सुरक्षा सुरक्षा उपाय, और OpenAI-शैली के एपीआई।

विस्तृत विवरण

HelixAgent एक उत्पादन-तैयार, AI-संचालित समूह LLM सेवा (MIT) है जो किसी एक मॉडल के उत्तर को परिकल्पना मानती है, निर्णय नहीं। यह किसी एक प्रदाता पर निर्भर नहीं रहती जो गलत, पक्षपाती या अस्थायी रूप से अनुपलब्ध हो सकता है, बल्कि कई भाषा मॉडलों के प्रतिक्रियाओं को संयोजित कर सबसे सटीक और विश्वसनीय आउटपुट पर पहुँचती है — और जब प्रश्न इतना जटिल हो कि उसका समाधान आवश्यक लगे, तो यह मॉडलों को एक संरचित, बहु-चरणीय बहस प्रक्रिया से गुजारती है। इसका प्रदाता समूह व्यापक है: इसके README में `internal/llm/providers/` के तहत कई LLM प्रदाताओं का उल्लेख है, जिनमें Claude, DeepSeek, Gemini, Mistral, Qwen, और xAI/Grok शामिल हैं।

महत्वपूर्ण बात यह है कि प्रदाता चयन कोई स्थिर प्राथमिकता सूची नहीं है — यह वास्तविक समय में अर्जित किया जाता है। एकीकृत LLMsVerifier से प्राप्त लाइव सत्यापन स्कोर रूटिंग और सर्वश्रेष्ठ प्रदर्शन करने वाले प्रदाता की ओर सहज पतन को संचालित करते हैं, साथ ही किसी प्रदाता के प्रदर्शन में गिरावट आने पर श्रेणीबद्ध त्रुटि रिपोर्टिंग भी की जाती है। AI बहस संचालक असहमति को संकेत में बदलता है: यह कई

टोपोलॉजी (जाल, तारा, श्रृंखला) और एक अनुशासित चरण प्रोटोकॉल — प्रस्ताव → आलोचना → समीक्षा → संश्लेषण — का समर्थन करता है, जिसमें क्रॉस-बहस सीखने की व्यवस्था है ताकि समय के साथ मॉडलों को समन्वित करने की प्रणाली की क्षमता में सुधार हो सके। रूटिंग रणनीतियों में आत्मविश्वास-भारित चयन, बहुमत-मत सहमति, और अर्थगत-आशय पहचान शामिल हैं, सभी वास्तविक समय में स्ट्रीमिंग प्रतिक्रियाओं के साथ ताकि उत्तर टोकन-दर-टोकन प्राप्त हों, न कि पूरे समूह के निर्णय के बाद।

यह सेवा उत्पादन में जीवित रहने के लिए इंजीनियर की गई है, न कि केवल प्रदर्शन के लिए: PostgreSQL और Redis एक उच्च-उपलब्धता डेटा परत का निर्माण करते हैं, Prometheus/Grafana/OpenTelemetry मेट्रिक्स, डैशबोर्ड और ट्रेसिंग प्रदान करते हैं, और JWT प्रमाणीकरण, दर सीमित करने, सुरक्षा उपायों, और PII पहचान के साथ समूह को वास्तविक तैनाती के लिए आवश्यक नियंत्रणों में लपेटते हैं। यह लगभग बीस अलग-अलग मॉड्यूल्स (इवेंटबस, ऑब्जर्वेबिलिटी, ऑथ, स्टोरेज, वेक्टरडीबी, एम्बेडिंग्स, RAG, मेमोरी, MCP, और अन्य) के रूप में संगठित है, जिनमें से प्रत्येक एक अलग चिंता का विषय है, और यह एक LLM अनुकूलन ढाँचा (अर्थगत कैशिंग, संरचित आउटपुट, उन्नत स्ट्रीमिंग) के साथ आता है, जिसमें SGLang, LlamaIndex, LangChain, Guidance, और LMQL के लिए एकीकरण शामिल हैं। चूंकि पूरा होने और समूह के एंडपॉइंट OpenAI-संगत हैं, इसलिए मौजूदा क्लाइंट HelixAgent की ओर इशारा कर सकता है और बिना किसी पुनर्लेखन के समूह तर्क प्राप्त कर सकता है।

हमने इसे क्यों बनाया

कोई भी एकल LLM गलत, पूर्वाग्रहित या अनुपलब्ध हो सकता है। HelixAgent को इसलिए बनाया गया ताकि एप्लिकेशन एक साथ कई मॉडलों से सलाह ले सकें, उनकी उत्तर की विश्वसनीयता को मापकर उनका मूल्यांकन कर सकें, और आवश्यकता पड़ने पर सहजता से विकल्प चुन सकें—इससे एकल प्रदाता पर निर्भरता की नाजुकता एक लचीले, आत्म-मूल्यांकन करने वाले समूह में बदल जाती है।

यह क्यों गेम-चेंजर है

यह मल्टी-मॉडल सहमति को संचालित करता है—“कई मॉडलों से पूछो और उनके उत्तरों में सामंजस्य बैठाओ” को तदर्थ स्क्रिप्ट से निकालकर एक उत्पादन सेवा में बदल देता है। एक प्रदाता को हार्ड-कोड करके उम्मीद करने के बजाय, टीमों को लाइव सत्यापन स्कोर द्वारा संचालित रूटिंग मिलती है, उन सवालों के लिए एक संरचित बहस प्रोटोकॉल जहाँ एक बार का उत्तर पर्याप्त नहीं होता, और उत्पादन-स्तरीय लचीलापन (उच्च उपलब्धता डेटा परत, पूर्ण अवलोकन क्षमता, और सुरक्षा उपाय) जो OpenAI-संगत API के पीछे होता है। इसका लाभ है बिना व्यवधान के अपनाने की सुविधा: एक नाजुक एकल प्रदाता निर्भरता एक लचीले, आत्म-मूल्यांकन करने वाले समूह में बदल जाती है, और मौजूदा क्लाइंट केवल एंडपॉइंट बदलकर इसमें स्विच कर सकते हैं—अपने कोड में बदलाव किए बिना।

क्या है नवाचारी

- संरचित बहु-चरणीय AI बहस जो मॉडल असहमति को एक संसाधन के रूप में उपयोग करती है: चुनने योग्य मेश/स्टार/चेन टोपोलॉजी, एक अनुशासित प्रस्ताव→आलोचना→समीक्षा→संश्लेषण प्रोटोकॉल, और समय के साथ संचित होने वाला क्रॉस-डिबेट लर्निंग।

- लाइव LLMsVerifier स्कोर से अर्जित गतिशील प्रदाता चयन—स्थिर प्राथमिकता सूची के बजाय समूह उस प्रदाता को रूट करता है जो वास्तव में इस समय बेहतर प्रदर्शन कर रहा होता है, और जब कोई प्रदाता कमजोर पड़ता है तो सहजता से विकल्प चुन लेता है।
- एक मूल Go LLM-अनुकूलन ढाँचा (सिमेंटिक केश, संरचित आउटपुट, उन्नत स्ट्रीमिंग) जो स्वतंत्र रूप से खड़ा होता है, जिसमें बाहरी अनुकूलक (SGLang, LlamaIndex, LangChain, Guidance, LMQL) की आवश्यकता पड़ने पर जोड़ा जा सकता है, न कि अनिवार्य रूप से।
- लगभग बीस अलग-अलग मॉड्यूल का एक मॉड्यूलर आर्किटेक्चर जो चिंताओं को अलग रखता है और बिग डेटा सुविधाओं जैसे वितरित मेमोरी और ज्ञान-ग्राफ स्ट्रीमिंग के द्वार खोलता है।

सबसे बड़ी तकनीकी चुनौतियाँ और उनका समाधान

- कई असमान प्रदाताओं में से चुनाव करना। प्रदाताओं की गुणवत्ता अलग-अलग होती है और समय के साथ बदलती रहती है, इसलिए कोई भी स्थिर रैंकिंग अगले दिन ही गलत साबित हो सकती है। हमने इसका समाधान निरंतर मापन द्वारा किया: LLMsVerifier स्कोर विश्वास-भारित और बहुमत-मत रूटिंग को संचालित करते हैं, जिसमें सहज फॉलबैक होता है ताकि प्रदर्शन में गिरावट आने पर प्रदाता को दरकिनार किया जा सके न कि उस पर भरोसा किया जाए।
- वास्तव में कठिन सवालों का विश्वसनीय उत्तर प्राप्त करना। एक बार पूछे जाने पर एकल मॉडल की अपनी गलती पकड़ने का कोई तंत्र नहीं होता। डिबेट ऑर्किस्ट्रेटर इसे प्रदान करता है—बहु-टोपोलॉजी, चरणबद्ध बहस (प्रस्ताव → आलोचना → समीक्षा → संश्लेषण) जो मॉडलों को एक-दूसरे को चुनौती देने और परिष्कृत करने के लिए बाध्य करती है, इससे पहले कि अंतिम उत्तर संश्लेषित किया जाए।
- एक समूह को नोटबुक से बाहर उत्पादन में चलाना। कई प्रदाताओं तक विस्तार करने से विफलता की सतह बढ़ जाती है। हमने इसे PostgreSQL+Redis उच्च उपलब्धता डेटा परत, Prometheus/Grafana/OpenTelemetry अवलोकन क्षमता (जब कोई प्रदाता या रूट गड़बड़ करता है), और एक सुरक्षा परिधि के माध्यम से नियंत्रित किया जिसमें JWT प्रमाणीकरण, दर सीमित करना, एक गार्डरेल इंजन, और PII पहचान शामिल है।

सामग्री

तकनीकी स्टैक

- **Go** — चुना गया क्योंकि एकल अनुरोध को एक साथ कई प्रदाताओं तक फैलाना ठीक वही काम है जिसके लिए गो-रूटीन बनाए गए हैं, और सिंगल-बाइनरी डिप्लॉयमेंट ~20-मॉड्यूल सेवा को शिप करने में सरलता बनाए रखता है; यह पूरी सेवा और हर आंतरिक मॉड्यूल की रीढ़ है।
- **Gin (वेब API)** — तेज़, कम-ओवरहेड HTTP इंटरफ़ेस के लिए चुना गया; यह OpenAI-संगत /v1 कम्पलीशन, चैट, स्ट्रीमिंग और एन्सेम्बल एंडपॉइंट्स को सेवा प्रदान करता है, जो मौजूदा क्लाइंट्स को बिना किसी बदलाव के एन्सेम्बल अपनाने की सुविधा देते हैं।
- **PostgreSQL** — सत्रों, विश्लेषण और बहस रिकॉर्ड्स के लिए स्थायी भंडारण के रूप में चुना गया, ताकि सहमति निर्णय और बहस इतिहास ऑडिट योग्य रहें; यह HA डेटा परत का आधार है।

- **Redis** — कम-विलंबता कैशिंग और टास्क क्यूइंग के लिए चुना गया; यह प्रतिक्रिया कैशिंग और सिमेंटिक कैश परत को शक्ति प्रदान करता है, जो बार-बार या लगभग समान प्रॉम्प्ट्स को अनावश्यक अनुमान से बचाता है।
- **LLMsVerifier (एकीकृत)** — प्रदाता विश्वसनीयता को एक मापी गई मात्रा बनाने के लिए चुना गया, न कि केवल एक धारणा; इसके स्कोर प्रदाताओं को रूटिंग के लिए रैंक करते हैं और किसी एक के प्रदर्शन में गिरावट आने पर फॉलबैक को संचालित करते हैं।
- **Prometheus + Grafana + OpenTelemetry** — चुना गया ताकि कई प्रदाताओं पर फैला एन्सेम्बल अवलोकन योग्य बना रहे; ये helixagent * मेट्रिक्स, डैशबोर्ड और फैन-आउट के पार एंड-टू-एंड अनुरोध ट्रेसिंग को उजागर करते हैं।
- **Model Context Protocol (MCP) एडेप्टर** — खुले प्रोटोकॉल के माध्यम से विस्तारशीलता के लिए चुना गया; README में बाहरी टूल्स और संदर्भ को जोड़ने के लिए कई MCP एडेप्टर सूचीबद्ध हैं।
- **Neo4j / ClickHouse / Kafka (बिगडेटा)** — एकल नोड की सीमाओं को पार करने के लिए चुना गया: Neo4j और ClickHouse वितरित मेमोरी और ज्ञान-ग्राफ़ सुविधाओं को समर्थन देते हैं, और Kafka उस ग्राफ़ और इवेंट डेटा को बड़े पैमाने पर स्ट्रीम करता है।
- **ऑप्टिमाइजेशन इंटीग्रेशन (SGLang, LlamaIndex, LangChain, Guidance, LMQL)** — उपसर्ग कैशिंग, पुनर्प्राप्ति, टास्क डिक्म्पोज़िशन और नियंत्रित पीढ़ी को वैकल्पिक सेवाओं के रूप में जोड़ने के लिए चुना गया, ताकि भारी ऑप्टिमाइजेशन उपलब्ध हो पर अनिवार्य न हो।

स्थिति और ईमानदारी संबंधी टिप्पणियाँ

- **स्थिति:** बीटा। सेवा को उत्पादन-तैयार बताया गया है, लेकिन README में दिए गए प्रदर्शन और कवरेज आँकड़े (जैसे “1000+ अनुरोध/सेकंड”, “<500ms कैशड”, प्रदाता और सत्यापन-स्क्रिप्ट की संख्या) परियोजना के स्वयं के दावे हैं, स्वतंत्र रूप से सत्यापित नहीं, और यहाँ जानबूझकर गुणात्मक रूप से रखे गए हैं।
- प्रदाताओं की संख्या README में ही भिन्न है; पृष्ठ गुणात्मक “कई प्रदाता” के ढाँचे का उपयोग करता है।

प्राथमिकता स्तर: Helix-प्राथमिक।